

Cahier des charges — EventHub

API REST de gestion d'événements collaboratifs

Formation	Développement Backend / Node.js
Type de projet	Evaluation / TP de reprise
Durée indicative	2 à 4 séances
Livrable	API REST fonctionnelle + script SQL + collection Postman

1. Contexte et objectif

Vous devez concevoir et développer une **API REST** permettant de gérer des **événements** (soirées, ateliers, conférences, etc.), d'y **associer des participants**, et de laisser des **commentaires**.

L'API doit être sécurisée : seul un utilisateur authentifié (via un **token JWT**) peut accéder aux ressources protégées.

Objectifs pédagogiques

À la fin de ce projet, vous serez capable de :

- Structurer une API Node.js / Express en couches (Routes, Controllers, Models, Middleware, Config)
- Mettre en place une authentification avec **inscription**, **connexion**, **hachage de mot de passe** (bcrypt) et **JWT**
- Protéger des routes via un **middleware** de vérification de token
- Interagir avec une base **MySQL** (CRUD, jointures, clés étrangères)
- Exposer une API testable via Postman

2. Stack technique imposée

Élément	Technologie
Runtime	Node.js (modules ES6 : "type": "module")
Framework	Express
Base de données	MySQL
Connexion BDD	mysql2/promise
Authentification	jsonwebtoken + bcrypt
Variables d'environnement	dotenv
Relance auto (dev)	nodemon

Dépendances attendues

```
{
  "dependencies": {
    "bcrypt": "^6.0.0",
    "dotenv": "^18.0.5",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.3",
    "mysql2": "^3.24.5"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

3. Architecture du projet

L'arborescence suivante est **imposée** :

```
eventhub/
├─ Config/
│   └─ db.js          # Connexion MySQL
├─ Controllers/
│   ├── auth.controller.js
│   ├── users.controller.js
│   ├── events.controller.js
│   ├── participants.controller.js
│   └─ comments.controller.js
├─ Middleware/
│   └─ auth.middleware.js  # Vérification du token JWT
├─ Models/
│   ├── users.model.js
│   ├── events.model.js
│   ├── participants.model.js
│   └─ comments.model.js
├─ Routes/
│   ├── auth.route.js
│   ├── users.route.js
│   ├── events.route.js
│   ├── participants.route.js
│   └─ comments.route.js
├─ .env
├─ .gitignore
├─ bdd.sql
├─ index.js
└─ package.json
```

Responsabilités des couches

Couche	Rôle
Routes	Déclarent les endpoints et appellent le controller

Controllers	Valident les entrées, gèrent les codes HTTP, appellent le model
Models	Contiennent uniquement les requêtes SQL
Middleware	Intercepte la requête (ex. : vérifier le JWT) avant le controller
Config	Centralise la connexion à la base de données

4. Base de données

4.1 Création

Créer une base nommée `eventhub` et un utilisateur MySQL dédié avec les droits `SELECT` , `INSERT` , `UPDATE` , `DELETE` .

4.2 Schéma

Table `users`

Colonne	Type	Contraintes
<code>us_id</code>	INT	PK, AUTO_INCREMENT
<code>us_username</code>	VARCHAR(255)	NOT NULL, UNIQUE
<code>us_password</code>	TEXT	NOT NULL
<code>us_email</code>	VARCHAR(255)	NOT NULL, UNIQUE

Table `events`

Colonne	Type	Contraintes
<code>ev_id</code>	INT	PK, AUTO_INCREMENT
<code>ev_title</code>	VARCHAR(255)	NOT NULL
<code>ev_description</code>	TEXT	
<code>ev_date</code>	DATE	
<code>ev_location</code>	VARCHAR(255)	
<code>ev_owner</code>	INT	NOT NULL, FK → <code>users(us_id)</code>

Table `comments`

Colonne	Type	Contraintes
<code>co_id</code>	INT	PK, AUTO_INCREMENT
<code>co_comment</code>	TEXT	NOT NULL
<code>co_user</code>	INT	NOT NULL, FK → <code>users(us_id)</code>
<code>co_event</code>	INT	NOT NULL, FK → <code>events(ev_id)</code>

Table participants

Table d'association **N-N** entre utilisateurs et événements.

Colonne	Type	Contraintes
pa_user	INT	PK composite, FK → users(us_id)
pa_event	INT	PK composite, FK → events(ev_id)

4.3 Script SQL attendu

Fournir un fichier `bdd.sql` contenant :

1. La création de la base
2. La création des 4 tables (avec contraintes)
3. La création de l'utilisateur MySQL + `GRANT` + `FLUSH PRIVILEGES`

5. Variables d'environnement

Fichier `.env` (à ne **jamais** committer) :

```
SERVER_PORT=3000
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=event_user
DB_PASSWORD=*****
DB_NAME=eventhub
JWT_SECRET=une_cle_secrete_longue
```

Le `.gitignore` doit au minimum exclure : `node_modules/` , `.env` .

6. Authentification et sécurité

6.1 Inscription — `POST /auth/register`

- Corps attendu : `{ "username", "email", "password" }`
- Le mot de passe doit être **haché avec bcrypt** avant insertion
- Réponse : `201` + résultat de l'insertion
- En cas d'erreur : `500`

6.2 Connexion — `POST /auth/login`

- Corps attendu : `{ "login", "password" }`
- Vérifier que le login et le mot de passe sont fournis
- Récupérer l'utilisateur par username
- Comparer le mot de passe avec `bcrypt.compareSync`
- Si OK : générer un **JWT** contenant les infos utilisateur, expiration **1 heure**
- Réponse : le token (chaîne JWT)
- Identifiants incorrects : `403`
- Erreur serveur : `500`

Les routes `/auth/*` restent **publiques** (pas de middleware).

6.3 Middleware `checkToken`

Créer un middleware qui :

1. Lit le header `Authorization`
2. Extrait le token au format `Bearer <token>`
3. Si absent → `401 { "error": "Unauthorized" }`
4. Vérifie le token avec `jwt.verify` et `JWT_SECRET`
5. Si invalide → `403 { "error": "Token incorrect" }`
6. Si valide → appelle `next()`

6.4 Protection des routes

Dans `index.js`, appliquer le middleware sur **toutes** les routes sauf `/auth` et la route de santé `/` :

```
/events      → protégées
/users       → protégées
/participants → protégées
/comments    → protégées
/auth        → publiques
/            → publique (status OK)
```

7. Spécification des endpoints

7.1 Santé de l'API

Méthode	URL	Auth	Description
GET	/	Non	Retourne { "status": "OK" }

7.2 Auth (`/auth`)

Méthode	URL	Auth	Description
POST	<code>/auth/register</code>	Non	Inscription
POST	<code>/auth/login</code>	Non	Connexion → retourne un JWT

7.3 Utilisateurs (`/users`) — protégées

Méthode	URL	Description
GET	<code>/users</code>	Liste tous les utilisateurs
GET	<code>/users/:id</code>	Détail d'un utilisateur
POST	<code>/users</code>	Créer un utilisateur
PATCH	<code>/users/:id</code>	Modifier username / email
DELETE	<code>/users/:id</code>	Supprimer un utilisateur

7.4 Événements (/events) — protégées

Méthode	URL	Description
GET	/events	Liste tous les événements
GET	/events/:id	Détail d'un événement avec jointure sur le propriétaire (username, email)
POST	/events	Créer un événement
PATCH	/events/:id	Modifier un événement (champs partiels)
DELETE	/events/:id	Supprimer un événement

Création (POST /events)

Corps attendu :

```
{
  "title": "Atelier Node.js",
  "description": "Introduction à Express et JWT",
  "date": "2026-11-15",
  "location": "Salle B12",
  "owner": 1
}
```

Règles :

- `title` et `owner` sont **obligatoires** → sinon `403`
- Succès → `201`

Modification (PATCH /events/:id)

- Si l'événement n'existe pas → `404`
- Mise à jour **partielle** : ne modifier que les champs fournis (`title` , `description` , `date` , `location`)
- Succès → message de confirmation + objet mis à jour

7.5 Participants (/participants) — protégées

Méthode	URL	Description
GET	/participants	Liste toutes les inscriptions
GET	/participants/user/:userID	Événements auxquels un user participe
GET	/participants/event/:eventID	Participants d'un événement
POST	/participants	Inscrire un user à un événement
DELETE	/participants/:userID/:eventID	Désinscrire un user

Corps du POST /participants

```
{
  "user": 2,
  "event": 1
}
```

Succès → 201

7.6 Commentaires (/comments) — protégées

Méthode	URL	Description
GET	/comments	Liste tous les commentaires
GET	/comments/:id	Détail d'un commentaire
POST	/comments	Ajouter un commentaire
PATCH	/comments/:id	Modifier le texte d'un commentaire
DELETE	/comments/:id	Supprimer un commentaire

Corps du POST /comments

```
{
  "comment": "Super événement, j'y serai !",
  "user": 1,
  "event": 1
}
```

Modification (PATCH /comments/:id)

- Commentaire inexistant → 404
- Corps: { "comment": "nouveau texte" }

8. Gestion des erreurs

Situation	Code HTTP
Succès lecture / update / delete	200
Création réussie	201
Token absent	401
Token invalide / identifiants incorrects / champs obligatoires manquants	403
Ressource introuvable	404
Erreur serveur / BDD	500

Toutes les réponses d'erreur doivent être au format JSON :

```
{ "error": "Message explicite" }
```

Chaque controller doit utiliser un bloc `try / catch` .

9. Point d'entrée `index.js`

Le fichier doit :

1. Charger `dotenv`
 2. Créer l'app Express
 3. Activer `express.json()`
 4. Monter les routes (avec middleware sur les routes protégées)
 5. Exposer `GET /` avec `{ status: "OK" }`
 6. Écouter sur `process.env.SERVER_PORT`
 7. Afficher en console les URL locales (`127.0.0.1` et `localhost`)
-

10. Livrables attendus

1. **Code source** complet selon l'architecture imposée
 2. **bdd.sql** : création BDD + tables + utilisateur MySQL
 3. **.env.example** (sans secrets) listant les variables nécessaires
 4. **Collection Postman** couvrant au minimum :
 - Register / Login
 - CRUD Events
 - CRUD Users
 - Participants (tous les endpoints)
 - Comments (tous les endpoints)
 - Appels avec et sans token (pour démontrer le middleware)
 5. **README.md** court : installation, lancement, exemples d'appels
-

11. Bonus (facultatif)

- Ne pas renvoyer le mot de passe hashé dans les réponses `GET /users`
 - Récupérer l'utilisateur connecté depuis le token décodé (plutôt que de le passer dans le body)
 - Soft delete ou cascade SQL documentée
 - Validation plus poussée des emails / dates
-

12. Scénario de démonstration suggéré

1. Créer 2 utilisateurs via `/auth/register`
2. Se connecter avec le premier → récupérer le token
3. Créer un événement (owner = user 1) avec le header `Authorization: Bearer <token>`
4. Inscrire le user 2 comme participant
5. Lister les participants de l'événement
6. Ajouter un commentaire
7. Modifier l'événement (PATCH partiel)
8. Appeler `GET /events` **sans** token → vérifier le `401`
9. Appeler avec un token invalide → vérifier le `403`

13. Contraintes et consignes

- **Interdit** : stocker les mots de passe en clair
- **Interdit** : hardcoder les identifiants BDD ou le secret JWT dans le code
- **Interdit** : mettre toute la logique SQL dans les controllers
- **Obligatoire** : requêtes préparées (`?`) pour éviter les injections SQL
- **Obligatoire** : répondre toujours en JSON
- Le projet doit démarrer avec `npm start` (script nodemon)

Document destiné aux étudiants — à réaliser de manière autonome à partir de ce cahier des charges.